PRE TEST DAN POST TEST PRAKTIKUM

PENGOLAHAN CITRA

(Dosen : *Rio Priantama S.T., M.T.I.*)

Modul 4

Nama : Muhammad Rizal Nurfirdaus

NIM : 20230810088

Kelas : TINFC-2023-04

TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS KUNINGAN

PRE-TEST

1. Jelaskan tentang rumus Ro = Ri + CR

   Menurut saya, rumus **Ro = Ri + CR** merupakan salah satu operasi dasar dalam pengolahan citra digital yang digunakan untuk memodifikasi nilai intensitas piksel. Pada rumus tersebut, **Ro (result output)** adalah nilai piksel setelah diproses, **Ri (result input)** adalah nilai piksel sebelum diproses atau citra asli, sedangkan **CR (constant rate)** adalah nilai konstanta yang ditambahkan ke setiap piksel. Dengan demikian, proses ini bekerja dengan cara menambahkan suatu nilai tetap ke seluruh piksel dalam citra sehingga menghasilkan citra yang lebih terang.

   Secara konsep, rumus ini berfungsi untuk meningkatkan kecerahan citra. Setiap piksel akan mengalami peningkatan nilai intensitas yang sama, sehingga bagian gambar yang sebelumnya gelap menjadi lebih jelas terlihat. Operasi ini sering digunakan sebagai tahap awal dalam preprocessing sebelum dilakukan proses lanjutan seperti filtering, deteksi tepi, atau segmentasi. Namun, karena penambahannya bersifat konstan dan tidak mempertimbangkan kondisi lokal piksel, metode ini tidak adaptif terhadap variasi intensitas dalam citra.

   Dalam penerapannya, terdapat batasan nilai piksel yang harus diperhatikan, yaitu biasanya berada pada rentang 0 hingga 255 untuk citra grayscale 8-bit. Jika hasil perhitungan melebihi nilai maksimum tersebut, maka nilai piksel harus dipotong atau dibatasi menjadi 255 agar tetap valid. Sebagai contoh, jika nilai awal piksel adalah 230 dan ditambahkan konstanta 50, maka hasilnya menjadi 280, tetapi karena melebihi batas, nilai tersebut akan disesuaikan menjadi 255.

   Secara keseluruhan, menurut saya rumus **Ro = Ri + CR** merupakan metode yang sederhana, cepat, dan mudah diimplementasikan untuk meningkatkan kecerahan citra. Meskipun demikian, penggunaannya perlu diperhatikan karena dapat menyebabkan citra menjadi terlalu terang dan kehilangan detail jika nilai konstanta yang digunakan terlalu besar.

POST-TEST

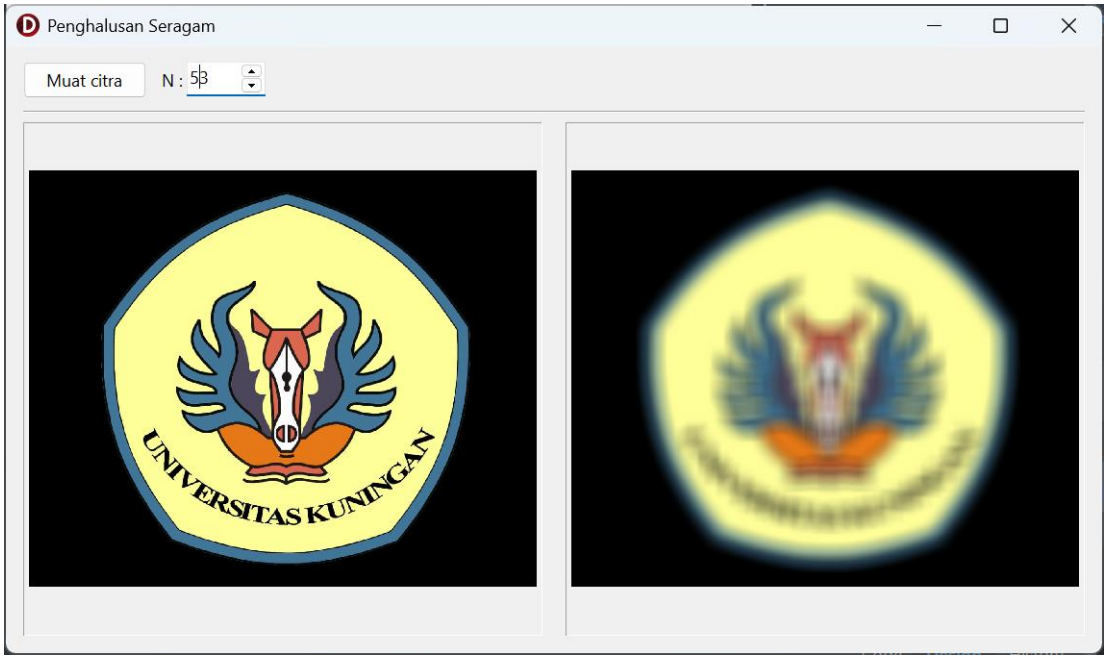
1. Buka Program Aplikasi Delphi
2. Buat proyek baru dan beri nama Penghalusan.dpr
3. Tambahkan Komponen-komponen berikut ke dalam Form1.

```
unit Unit2;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils, System.Variants,
System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs, Vcl.StdCtrls, Vcl.ExtCtrls, Vcl.Samples.Spin,
  Vcl.Imaging.jpeg, Vcl.Imaging.pngimage, Vcl.Imaging.GIFImg, System.Math;

type
  TForm2 = class(TForm)
    Button1: TButton;
    Label1: TLabel;
    SpinEdit1: TSpinEdit;
    Image1: TImage;
    Image2: TImage;
    OpenPictureDialog1: TOpenDialog;
    Bevel1: TBevel;
    Bevel2: TBevel;
    Bevel3: TBevel;
    procedure Button1Click(Sender: TObject);
    procedure SpinEdit1Change(Sender: TObject);
  private
    FUpdatingN: Boolean;
    procedure ApplyMeanFilter;
  public
  end;

var
  Form2: TForm2;

implementation

{$R *.dfm}

type
  PRGBTripleArray = ^TRGBTripleArray;
  TRGBTripleArray = array[0..(MaxInt div SizeOf(TRGBTriple)) - 1] of TRGBTriple;

procedure TForm2.Button1Click(Sender: TObject);
```

```
begin
  if OpenPictureDialog1.Execute then
  begin
    Image1.Picture.LoadFromFile(OpenPictureDialog1.FileName);
    ApplyMeanFilter;
  end;
end;

procedure TForm2.SpinEdit1Change(Sender: TObject);
begin
  if FUpdatingN then
    Exit;

  if SpinEdit1.Value mod 2 = 0 then
  begin
    FUpdatingN := True;
    try
      SpinEdit1.Value := SpinEdit1.Value + 1;
    finally
      FUpdatingN := False;
    end;
  end;

  if Assigned(Image1.Picture.Graphic) and (not Image1.Picture.Graphic.Empty) then
    ApplyMeanFilter;
end;

procedure TForm2.ApplyMeanFilter;
var
  SrcBmp, DstBmp: TBitmap;
  X, Y, N, Radius: Integer;
  W, H, Stride, Area: Integer;
  X1, Y1, X2, Y2: Integer;
  SrcRow: PRGBTripleArray;
  DstRow: PRGBTripleArray;
  SumR, SumG, SumB: Int64;
  IntegralR, IntegralG, IntegralB: array of Int64;

  function Idx(AX, AY: Integer): Integer;
  begin
    Result := (AY * Stride) + AX;
  end;
begin
  if (not Assigned(Image1.Picture.Graphic)) or Image1.Picture.Graphic.Empty then
    Exit;
```

```
N := SpinEdit1.Value;
if N < 1 then
  N := 1;

Radius := N div 2;

Screen.Cursor := crHourGlass;
SrcBmp := TBitmap.Create;
DstBmp := TBitmap.Create;
try
  SrcBmp.Assign(Image1.Picture.Graphic);
  SrcBmp.PixelFormat := pf24bit;

  W := SrcBmp.Width;
  H := SrcBmp.Height;
  if (W = 0) or (H = 0) then
    Exit;

  DstBmp.SetSize(W, H);
  DstBmp.PixelFormat := pf24bit;

  Stride := W + 1;
  SetLength(IntegralR, Stride * (H + 1));
  SetLength(IntegralG, Stride * (H + 1));
  SetLength(IntegralB, Stride * (H + 1));

  for Y := 1 to H do
  begin
    SrcRow := SrcBmp.ScanLine[Y - 1];
    for X := 1 to W do
    begin
      IntegralR[Idx(X, Y)] := IntegralR[Idx(X - 1, Y)] + IntegralR[Idx(X, Y - 1)] -
        IntegralR[Idx(X - 1, Y - 1)] + SrcRow[X - 1].rgbtRed;
      IntegralG[Idx(X, Y)] := IntegralG[Idx(X - 1, Y)] + IntegralG[Idx(X, Y - 1)] -
        IntegralG[Idx(X - 1, Y - 1)] + SrcRow[X - 1].rgbtGreen;
      IntegralB[Idx(X, Y)] := IntegralB[Idx(X - 1, Y)] + IntegralB[Idx(X, Y - 1)] -
        IntegralB[Idx(X - 1, Y - 1)] + SrcRow[X - 1].rgbtBlue;
    end;
  end;

  for Y := 0 to H - 1 do
  begin
    DstRow := DstBmp.ScanLine[Y];
    Y1 := Max(0, Y - Radius);
    Y2 := Min(H - 1, Y + Radius);
```

```
      for X := 0 to W - 1 do
      begin
        X1 := Max(0, X - Radius);
        X2 := Min(W - 1, X + Radius);

        Area := (X2 - X1 + 1) * (Y2 - Y1 + 1);

        SumR := IntegralR[Idx(X2 + 1, Y2 + 1)] - IntegralR[Idx(X1, Y2 + 1)] -
          IntegralR[Idx(X2 + 1, Y1)] + IntegralR[Idx(X1, Y1)];
        SumG := IntegralG[Idx(X2 + 1, Y2 + 1)] - IntegralG[Idx(X1, Y2 + 1)] -
          IntegralG[Idx(X2 + 1, Y1)] + IntegralG[Idx(X1, Y1)];
        SumB := IntegralB[Idx(X2 + 1, Y2 + 1)] - IntegralB[Idx(X1, Y2 + 1)] -
          IntegralB[Idx(X2 + 1, Y1)] + IntegralB[Idx(X1, Y1)];

        DstRow[X].rgbtRed := Byte(SumR div Area);
        DstRow[X].rgbtGreen := Byte(SumG div Area);
        DstRow[X].rgbtBlue := Byte(SumB div Area);
      end;
    end;

    Image2.Picture.Assign(DstBmp);
  finally
    Screen.Cursor := crDefault;
    SrcBmp.Free;
    DstBmp.Free;
  end;
end;

end.
```



Analisis : Menurut saya, kode yang telah saya buat sudah menunjukkan implementasi yang baik dalam menerapkan **mean filter (penghalusan seragam)** pada citra digital. Program ini terdiri dari dua bagian utama, yaitu proses pemuatan citra dan proses filtering. Ketika tombol "Muat citra" ditekan, citra ditampilkan pada komponen Image1, kemudian langsung diproses oleh prosedur ApplyMeanFilter dan hasilnya ditampilkan pada Image2. Nilai parameter **N** yang diinput melalui SpinEdit digunakan sebagai ukuran kernel, dan dijaga agar selalu bernilai ganjil sehingga memiliki titik pusat yang jelas dalam proses perhitungan.

Menurut saya, bagian paling penting dari kode ini terletak pada penggunaan **integral image**. Teknik ini memungkinkan perhitungan jumlah nilai piksel dalam suatu area dilakukan dengan sangat efisien tanpa harus melakukan perulangan bersarang untuk setiap piksel. Tiga array integral dibuat untuk masing-masing kanal warna, yaitu merah, hijau, dan biru. Dengan pendekatan ini, proses perhitungan rata-rata pada setiap window menjadi jauh lebih cepat, bahkan ketika ukuran kernel cukup besar seperti N = 53.

Dalam proses filtering, untuk setiap piksel ditentukan area tetangga berdasarkan nilai radius (N/2). Kemudian jumlah nilai piksel dalam area tersebut dihitung menggunakan integral image, dan hasilnya dibagi dengan luas area untuk mendapatkan nilai rata-rata. Nilai ini kemudian disimpan sebagai piksel baru pada citra hasil. Pendekatan ini menurut saya sudah sangat optimal dibandingkan metode konvensional yang menghitung rata-rata secara langsung menggunakan nested loop.

Jika dilihat dari hasil gambar, menurut saya efek yang dihasilkan sudah sesuai dengan teori mean filter. Citra hasil terlihat sangat halus dan kabur karena nilai N yang digunakan cukup besar. Detail seperti tulisan dan bentuk objek menjadi tidak jelas karena setiap piksel merupakan hasil rata-rata dari area yang luas. Hal ini menunjukkan bahwa mean filter bekerja dengan cara menghaluskan citra, namun konsekuensinya adalah hilangnya detail dan ketajaman tepi.

Secara keseluruhan, menurut saya kode yang telah saya buat sudah berhasil mencapai tujuan dari pengolahan citra, yaitu melakukan smoothing menggunakan mean filter. Implementasi yang digunakan juga sudah efisien dan terstruktur dengan baik, serta hasil yang diperoleh sesuai dengan karakteristik teori yang dipelajari.